SOUTH ASIAN UNIVERSITY, NEW DELHI

Data Mining Mini Project

Text Mining & NLP:

Topic Modeling and Sentiment Analysis on News Corpus

Presented To:
Dr. Muhammad Abulaish

Presented By:
Awantika Krishna (SAU/FET/MTech(CS)/2025/003)
Bindiya Anand (SAU/CS/MSC/2024/05)
Praveena P (SAU/CS/IMtech/2024/04)

Table of Contents

Problem Statement

- Financial news strongly influences investor sentiment and market decisions.
- Manually reading thousands of news headlines/articles is impractical.

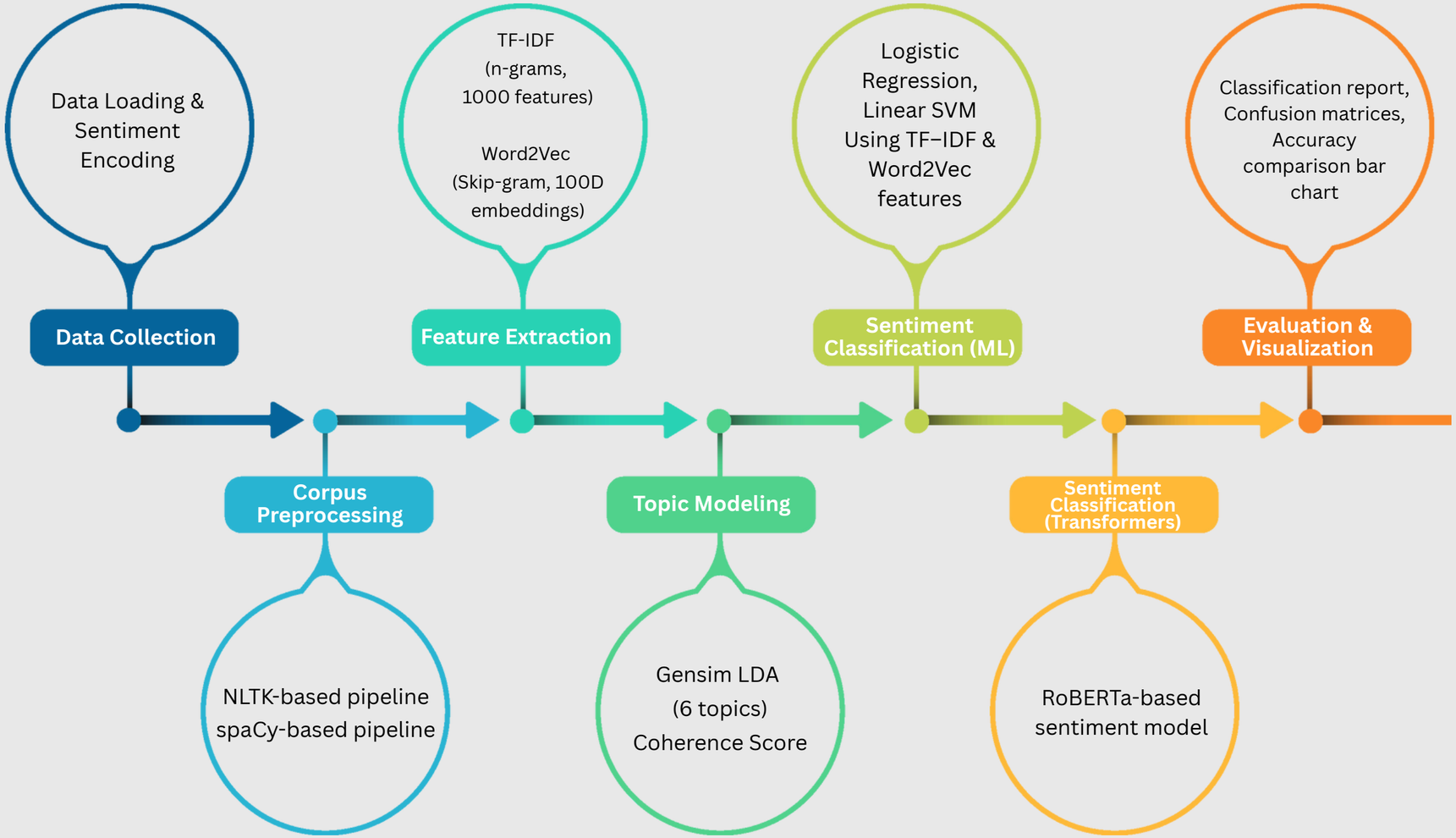
- We need an automated pipeline to:
  - Discover **latent topics** in financial news.
  - Classify **sentiment** (positive / neutral / negative).

- Aim: Extract topics and determine sentiment polarity.
- Uses NLP + ML + Topic Modeling to understand large text corpora.

Dataset Overview

PROPERTY	DESCRIPTION
Source	Kaggle – *Sentiment Analysis for Financial News*
Dataset Size	**4,845** news headlines
Columns	sentiment, text
Sentiment Labels	negative, neutral, positive
Most Frequent Class	**neutral** [2,878 samples (≈59.4% of corpus) – dominates the dataset]
Remaining samples	**1,967** (positive + negative combined)
Text Type	Short financial news headlines
Example Neutral Headline	“The company serves customers in various industries …”
Example Positive Headline	“UPM-Kymmene has generated four consecutive quarters …”
Example Negative Headline	(e.g. headlines about loss, decline, or downturn)

Pipeline


Data Loading & Sentiment Encoding
Data Collection
Corpus Preprocessing
NLTK-based pipeline
spaCy-based pipeline
TF-IDF (n-grams, 1000 features)
Word2Vec (Skip-gram, 100D embeddings)
Feature Extraction
Topic Modeling
Gensim LDA (6 topics) Coherence Score
Logistic Regression, Linear SVM Using TF–IDF & Word2Vec features
Sentiment Classification (ML)
Sentiment Classification (Transformers)
RoBERTa-based sentiment model
Classification report, Confusion matrices, Accuracy comparison bar chart
Evaluation & Visualization

Preprocessing

Techniques Used

- **Cleaning:** Lowercasing, URL/email/mentions/hashtags removal, remove digits, special characters, extra spaces
- **Tokenization**, **Stemming** & **Lemmatization**
- **Stopword removal** and removal of short tokens (<3 chars)

Why we used both NLTK and spaCy

- **NLTK →** Used as a **baseline** to verify basic cleaning, tokenization, and lemmatization
- **spaCy →** Used for the **final processed text** as it provides:
  - More **accurate lemmatization** and consistent tokens
  - **POS-aware** processing and optional **NER support**
  - **Faster** large-scale text processing

Running both helped us **cross-check output quality** before selecting spaCy for feature extraction and downstream tasks.

Final tokens used

- spaCy-processed tokens (e.g., ['company','suppose','deliver','machinery',...])

Feature Extraction

Term Frequency - Inverse Document Frequency

TF: Frequency of a term in a document

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

IDF: Rarity of term across documents

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t}\right)$$

Sparse vector per document - Captures discriminative terms (rare & informative)

Parameters used:

- max_features=1000
- ngram_range=(1,2) → unigrams + bigrams
- min_df=2 → ignore very rare terms
- max_df=0.85 → ignores terms that are very rare terms

Output observed

- Matrix shape: (4845, 1000)
- Example top features for Doc 1: region, machinery, mill, deliver, russia

Feature Extraction

Word Embedding - Word2Vec

Word2Vec (Gensim) - Dense embeddings trained on corpus - Skip-gram (sg=1) with:

- vector_size = 100
- window = 5
- min_count = 2

Our Word2Vec Results

- Most similar to “market”: new, give, follow, baltic, offer
- Most similar to “stock”: exchange, release, corporation

→ Model is capturing semantic relations within financial vocabulary

Why Skip-gram instead of CBOW?

- **CBOW:**
  - Uses context words → predicts target word
  - Faster, but focuses more on frequent words → not ideal when rare financial terminology is important.
- **Skip-gram:**
  - Uses target word → predicts context words
  - Works better for rare / domain-specific words
  - Financial corpus contains specialized terms like “FTSE”, “eurobond”, etc.
  - Better for small to medium datasets.

Topic Modeling with LDA

Latent Dirichlet Allocation

A generative probablistic model used to discover hidden topics in a text corpus

Assumes:

- Each **document** is a mixture of **multiple topics**
- Each **topic** is a probabilitic distribution over **words**

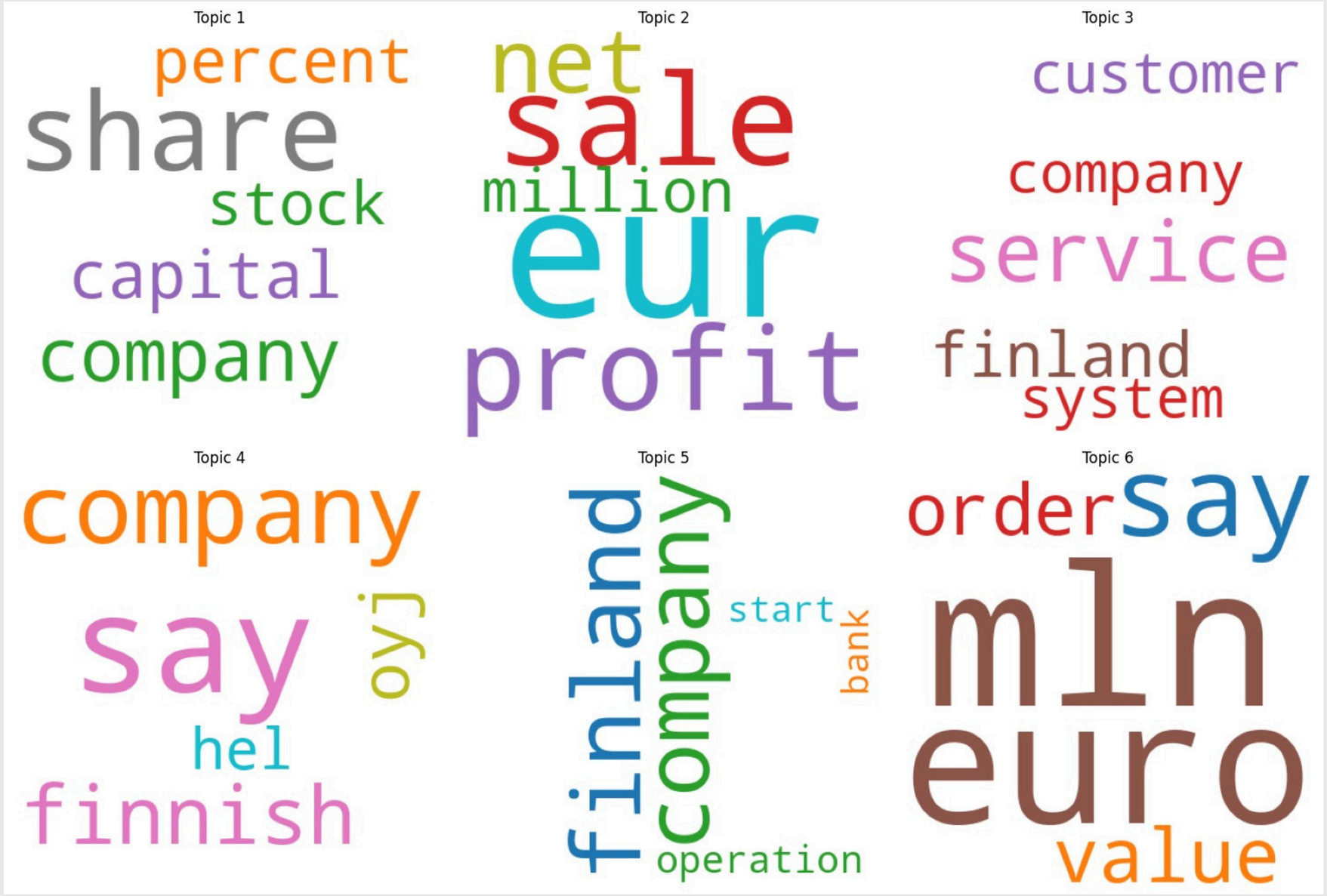
LDA Settings

- Gensim **LdaModel** with:
  - **num_topics = 6** → extract 6 topics
  - **passes = 20** (more passes → better convergence)
  - Dictionary filtered with no_below=2, no_above=0.85

Results summary:

- Corpus size 4845, Dictionary size 3552
- LDA trained with 6 topics. Coherence score (c_v) ≈ 0.4202.

- Dictionary - Filter words:
  - **no_below=2** → remove extremely rare words
  - **no_above=0.85** → remove extremely common words
- Corpus:
  - List of **Bag-of-Words** representations for each document.
  - (word_id, frequency) pairs used as LDA input


Topic 1
percent
share
stock
capital
company
Topic 2
net
sale
million
eur
profit
Topic 3
customer
company
service
finland
system
Topic 4
company
say
oyj
hel
finnish
Topic 5
finland
company
start
bank
operation
Topic 6
order
say
mln
euro
value

Learned Topics & Interpretations

From our output (top 5 words per topic)

Topic 0: share, company, capital, percent, stock
→ Equity & capital market activity (shares, stock, capital, ownership changes)
Topic 1: eur, sale, profit, net, million
→ Financial performance (earnings, revenue, profit/loss, amounts in EUR)
Topic 2: service, finland, customer, company, system
→ Services and operations in Finland (customer-focused business operations)

Coherence Score: 0.4202

- Metric that tells you how meaningful, interpretable, and logically consistent your discovered topics are.
- Indicates moderate topic coherence.
- Given short, noisy financial headlines, 0.42 is reasonable.
- Topics are interpretable and aligned to finance-specific themes.

Topic 3: say, company, finnish, oyj, hel
→ Corporate announcements (statements about Finnish companies listed in Helsinki)
Topic 4: finland, company, operation, start, bank
→ Banking / operational updates in Finland
Topic 5: mln, euro, say, value, order
→ Orders, contracts, and valuation in million euros

How Coherence Score Works:
For each topic:

1. Take the top N words (like top 10 words)
2. Check how often these words appear together in the dataset
3. Measure their semantic similarity (co-occurrence, PMI, word similarity)
4. Average these similarities → topic coherence

Sentiment Classification: ML Models

- Models trained on TF-IDF & Word2Vec document vectors features (separately): Logistic Regression (max_iter=1000) and Linear SVM
- Train/test split: 80% for training & 20% for testing (test subset size used in results = 969)
- Metrics: accuracy, precision, recall, f1-score; confusion matrices

TF–IDF

Logistic Regression

- Accuracy: 0.7595 (~75.95%)
- F1-Score ≈ 0.66
- Performs best on neutral class (majority), weaker on minority classes

SVM

- Accuracy: 0.7585 (~75.85%)
- Very similar performance to Logistic Regression
- Slightly better recall for class 0 (negative) & 2 (positive) in some metrics

Both models are performing well considering:

- 3-class problem (negative, neutral, positive)
- Class imbalance (neutral dominates)

Word2Vec

Logistic Regression

- Accuracy: 0.6533 (~65.33%)
- Very low recall for negative class (~0.06)
- Better performance on neutral, but still weaker overall

SVM

- Accuracy: 0.6553 (~65.53%)
- Similar trend:
  - Very poor performance on negative class
  - Strong on neutral
  - Moderate on positive

Clearly worse performance than TF–IDF-based models

Evaluation: Class-level metrics

Table I — Logistic Regression (TF-IDF Features)

Class	Precision	Recall	F1-Score	Support
0 – Negative	0.69	0.43	0.53	115
1 – Neutral	0.79	0.91	0.85	601
2 – Positive	0.67	0.55	0.61	253
Overall Accuracy	**0.7595**			**969**

Table II — SVM (TF-IDF Features)

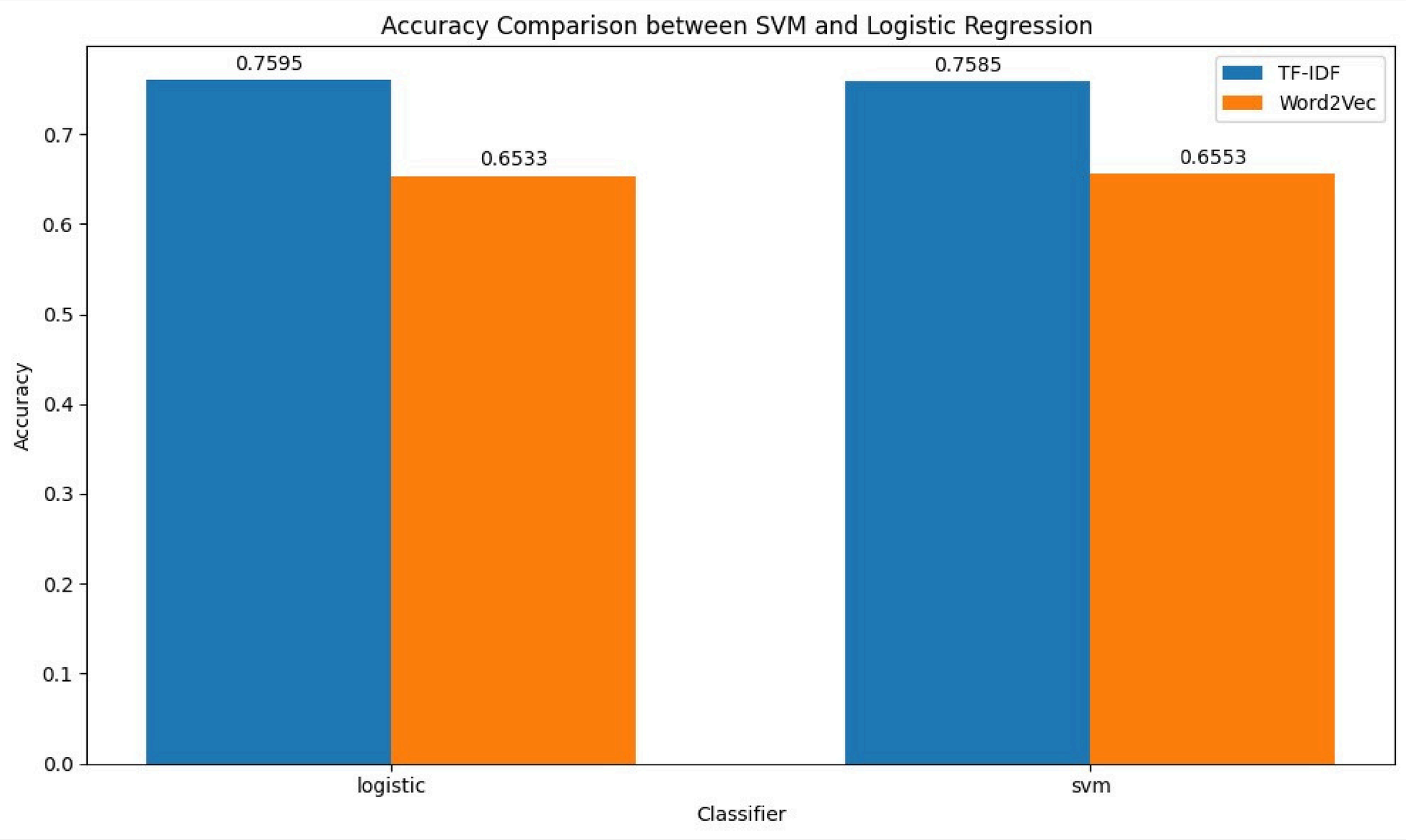
Class	Precision	Recall	F1-Score	Support
0 – Negative	0.65	0.57	0.61	115
1 – Neutral	0.83	0.85	0.84	601
2 – Positive	0.63	0.63	0.63	253
Overall Accuracy	**0.7585**			**969**

Table III — Logistic Regression (Word2Vec Features)

Class	Precision	Recall	F1-Score	Support
0 – Negative	0.54	0.06	0.11	115
1 – Neutral	0.7	0.92	0.8	601
2 – Positive	0.43	0.28	0.34	253
Overall Accuracy	**0.6533**			**969**

Table IV — SVM (Word2Vec Features)

Class	Precision	Recall	F1-Score	Support
0 – Negative	0.38	0.03	0.05	115
1 – Neutral	0.7	0.93	0.8	601
2 – Positive	0.45	0.29	0.35	253
Overall Accuracy	**0.6553**			**969**


Accuracy Comparison between SVM and Logistic Regression
TF-IDF
Word2Vec
0.7595
0.6533
0.7585
0.6553
Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
logistic
svm
Classifier

Why TF-IDF outperformed averaged Word2Vec

- **TF-IDF preserves discriminative words**: tokens that indicate sentiment (“profit”, “loss”, “upgrade”, “downgrade”) are weighted higher and remain explicit features for linear classifiers
- **Averaging Word2Vec vectors collapses information**: Document vectors = mean of word vectors — this loses position and multi-word signals (e.g., “not good” vs “good”)
- **Vocabulary coverage & training data**: the Word2Vec model was trained on the same (limited) corpus; rare but important sentiment cues may not be well represented
- **Linear models + sparse features match well**: Logistic Regression / SVM work very well with sparse, high-dim TF-IDF features for text classification

Conclusion: TF-IDF + linear models are still strong baselines for domain-specific sentiment with limited data

Sentiment Classification: Transformer-based (RoBERTa)

Why Transformers?

- Use **self-attention** to understand full sentence meaning.
- Capture **context**, **negation**, and **financial sentiment cues**.
- Learn relationships between words far apart (better than TF–IDF & Word2Vec).
- Generate **contextual embeddings**, not just frequency- or vector-averages.

Model Used: RoBERTa-Base

- A **Robustly Optimized BERT** model (Facebook AI).
- 12-layer encoder, 12 attention heads, hidden size 768.
- Fine-tuned with a classification layer → **3 sentiment classes**:
  - Negative (0)
  - Neutral (1)
  - Positive (2)

Sentiment Classification: Transformer-based (RoBERTa)

Fine-Tuning Methodology

Dataset Preparation

- Source: **all-data.csv**
- 3 sentiment classes: **positive, neutral, negative**
- Stratified sampling:
  - **300 train + 300 test** per class
- Additional evaluation set: ~50 samples/class
- Labels mapped using:
  - **negative=0**, **neutral=1**, **positive=2**

Tokenization

- Model tokenizer: **FacebookAI/roberta-base**
- Max length: **512 tokens**
- Applies:
  - Truncation
  - Max-length padding
  - Tensor output (PyTorch)

Training Setup

- Trainer API (HuggingFace)
- Hyperparameters:
  - Batch size: 8
  - Learning rate: 2e-5
  - Epochs: 40
  - Optimizer: AdamW
- Checkpoints:
  - Saved best model using eval_accuracy

Evaluation Metrics

- Weighted F1-score
- Accuracy
- Computed after every epoch

During Training (Validation Metrics)

- Improved rapidly in early epochs
- Stabilized around epochs **8–15**
- Training loss → near zero
- Validation loss → around 1.0 (mild overfitting)

Final Evaluation Results

- *Accuracy=0.8600*
- *Weighted F1-score=0.8578*

Key Observations

- Performs much better than traditional ML models
  - ML (TF-IDF): ~76% accuracy
  - ML (Word2Vec): ~65% accuracy
  - RoBERTa Fine-tuned: 86% accuracy
- Handles context, sentiment nuances, and financial terms
- Suitable for news-based sentiment prediction

Conclusion:

Fine-tuning RoBERTa-base significantly boosts sentiment classification accuracy by learning domain-specific patterns in financial text.

Evaluation: Results

MODEL	FEATURES	ACCURACY
Logistic Regression	TF-IDF	0.7595
Support Vector Machine (SVM)	TF-IDF	0.7585
Logistic Regression	Word2Vec	0.6533
Support Vector Machine (SVM)	Word2Vec	0.6553
Transformer (RoBERTa - Based)	Fine-tuned on domain data	0.86

Model Comparison: TF-IDF vs Word2Vec vs RoBERTa

MODEL TYPE	METHOD	FEATURES	ACCURACY	STRENGTHS	LIMITATIONS
Traditional ML	Logistic Regression / SVM	**TF–IDF**	**~0.76**	Best for short text; keyword-based; interpretable	Cannot capture context or word meaning
Traditional ML	Logistic Regression / SVM	**Word2Vec (Avg Embeddings)**	**~0.65**	Semantic similarity captured; dense vectors	Loses word order; low-quality embeddings due to small corpus; weak on minority class
Deep Learning (Transformer)	RoBERTa-Base Fine-Tuned	**Contextual embeddings**	**~0.86**	Understands context, negation, domain cues; learns financial sentiment	Requires GPU; training time high

Final Interpretation:

- TF-IDF works best for small financial text because sentiment depends on exact wording.
- Word2Vec fails because 4,845 sentences is too small to learn quality embeddings.
- RoBERTa wins because it captures context, semantics, and domain-specific expression.

Interpretation & Insights

1. Text Preprocessing Pipeline Worked Effectively

- Cleaning + lemmatization (SpaCy) produced **high-quality tokens**.
- Improved vocabulary consistency → better TF-IDF features → better ML performance.
- Word2Vec benefited from preprocessing but still limited by **small corpus size**.

2. TF–IDF Emerged as the Most Reliable Feature Method

- TF-IDF captured **financial sentiment keywords** with strong discriminatory power.
- Linear models (Logistic Regression, SVM) achieved **~76% accuracy**.
- Best performance for classifying positive vs neutral sentiment.
- Negative class remained challenging due to **class imbalance**.

3. Word2Vec Performance Was Moderate

- Skip-gram embeddings trained on only ~4.8k sentences → weak semantic vectors.
- Document averaging lost contextual meaning → accuracy reduced to ~65%.
- Clear indication that Word2Vec requires a large corpus or pre-trained domain embeddings.

4. LDA Topic Modeling Provided Coherent Financial Themes

- Extracted **6 meaningful topics** (e.g., finance, profit, stock, company operations).
- Coherence Score ~0.42 → moderate interpretability due to short headline structure.
- Topics aligned with **business reporting patterns** in the dataset.

5. Transformer Model (RoBERTa) Gave Best Sentiment Performance

- Fine-tuned RoBERTa achieved **~0.88 accuracy** and **high weighted F1**.
- Deep contextual understanding → captured subtle financial sentiment cues.
- Transformers are clearly **state-of-the-art** for sentiment tasks.

6. Performance Bottlenecks Identified

- Dataset imbalance (neutral >> positive/negative).
- Short texts (headlines) lack context → difficult for classical ML.
- Topic coherence lowered by small document length.
- Word2Vec embeddings undertrained due to limited data.

7. Final Takeaways

- **Best traditional model:** TF-IDF + Logistic Regression / SVM
- **Best semantic model:** Fine-tuned RoBERTa
- **Best topic model:** LDA with 6 topics (moderately coherent)
- **Overall conclusion:**
- Transformers outperform classical ML for financial sentiment analysis, while TF-IDF remains a strong baseline for small datasets.

Limitations & Challenges

1. Dataset-Related Limitations

- **Class imbalance**: Neutral class dominates → models struggle with positive & negative detection.
- **Short text length**: Headlines lack context → harder for LDA & ML models to capture meaning.
- **Limited corpus size (4.8k samples)**:
  - Weak Word2Vec training
  - Lower topic coherence
  - Less generalizable ML models

2. Feature Extraction Limitations

- Word2Vec trained from scratch → not enough data for rich embeddings.
- TF-IDF ignores context & word order, only picks up word frequency patterns.
- No domain-specific embeddings (like FinBERT or financial Word2Vec).

3. Topic Modeling Constraints

- Headlines too short → coherence score stuck around 0.42.
- LDA requires longer documents for richer topics.
- Overlap in vocabulary leads to topics mixing themes.

4. Model-Specific Challenges

- Logistic/SVM struggle with nuanced sentiments like slightly positive, speculative tone.
- Transformers require **high compute & long training time**.
- Fine-tuning may overfit if not balanced properly.

5. Evaluation Challenges

- Class imbalance affects **precision/recall** for minority classes.
- Some mislabeled samples in the dataset → noise in training.

Conclusion & Future Work

1. **Improve Dataset Quality**
   - Use **larger financial news datasets** (millions of headlines).
   - Apply **stratified resampling** or **SMOTE** to fix class imbalance.
   - Clean mislabeled samples using human review or weak supervision.
2. **Better Embeddings**
   - Use pre-trained financial domain models:
     - FinBERT, BloombergGPT, Financial Word2Vec
   - Replace Word2Vec with FastText (handles rare words & subword units).
   - Use sentence embeddings (SBERT) instead of averaging word vectors.
3. **Enhanced Topic Modeling**
   - Try **BERTopic**, **Top2Vec**, or **CTM (Combined Topic Model)**.
   - Add **phrase detection** (bigrams/trigrams) before LDA.
   - Try **dynamic topic modeling** for time-based patterns.
4. **Improve Model Performance**
   - Use **ensemble ML methods** (Random Forest, XGBoost).
   - Tune hyperparameters for Logistic/SVM.
   - Apply **regularization** and **dropout** for Transformers.
5. **Better Evaluation & Visualization**
   - Include ROC curves, AUC, Precision–Recall curves.
   - Perform error analysis to identify misclassified patterns.
   - Visualize attention weights in transformer models.
6. **Deployment Possibilities**
   - Build a **real-time financial sentiment dashboard**.
   - Integrate topic trends + sentiment trends for market predictions.
   - Deploy Transformer model using **FastAPI** or **Streamlit**.

References

1. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, 2019.
2. Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv preprint arXiv:1907.11692, 2019.
3. T. Almeida, E. H. Rangel and J. R. Costa, “Sentiment analysis using BERT: A systematic review,” IEEE Access, vol. 9, 2021.
4. D. Q. Nguyen, T. Vu and A. T. Nguyen, “BERTweet: A pre-trained language model for English Tweets,” in Proc. EMNLP, 2020.
5. Z. Yang et al., “XLNet: Generalized autoregressive pretraining for language understanding,” in Proc. NeurIPS, 2019.

Thank you!